

Dungeon Surge

Abschlusspräsentation

2D Wave-Survival mit Bosskämpfen, Upgrades und Level-Fortschritt

Bilgehan Fabian Selim Berk Tan

Projektpräsentation

30. Juni 2026

Kurzüberblick über Team und Rollen

Bilgehan

Skripte, Level 1 und Level 2 Design, Gegnerdesign, Player-Design, Player-Animationen und zentrale Gameplay-Logik.

Fabian

Unterstützung bei Scripts, Boss-Systemen, Level-up-Design, allgemeiner Gameplay-Hilfe, Upgrade-System und zwei Bosse.

Selim Berk Tan

UI-Elemente wie Victory Screen und Endscreen, Soundeffekte, Audio-Integration und Game-End-Management.

Finales Konzept und Designentscheidungen

- Ziel: ein klar verständliches 2D-Action-Spiel.
- Fokus auf kurze Core-Loop aus Kampf, XP und Upgrades.
- Zwei Level für sichtbare Progression und Steigerung.
- Bosskämpfe als klare Hohepunkte jeder Spielphase.



Visuelle Designentscheidungen mit echten Assets



- Einheitlicher Fantasy-Look für Menu und HUD.
- Holz-, Stein- und Pergament-Optik passend zur Spielwelt.
- Große Buttons und klare Kontraste für gute Lesbarkeit.
- Deutlicher Victory-Screen als starker Abschluss.

Tatsächlich verwendete Tools und Software

- Engine: Unity mit 2D-Physik, Animation, Prefabs und Szenenverwaltung.
- Programmiersprache: C# für Gameplay-, UI-, Audio- und Progressionslogik.
- Projektstruktur: Szenen in Assets/Scenes, Skripte in Assets/Scripts, Gameplay-Prefabs in Assets/Prefabs.
- UI-Assets: vorhandene Screens für Main Menu, Options, Level-Screens und Victory-Screens in Assets/UIs.
- Audio-Assets: Boss-Spawn, Boss-Fight, Treffer-, Gold-, Schwert- und weitere Sounds in Assets/Audio.

Projektaufbau im Code

Szenen

- MainMenu
- Level1
- Level2

Wichtige Prefabs

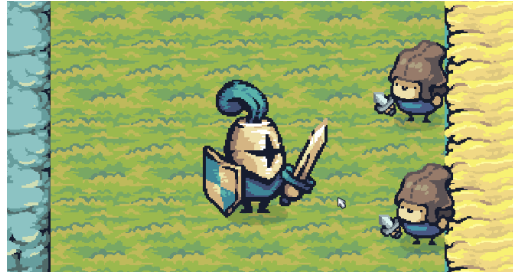
- Pawn
- Lancer
- Boss_GoblinKing
- Boss_VampireBat
- Gold
- BossRockProjectile

Zentrale Skripte

- WaveManager
- LevelManager
- UpgradeSelectionManager
- BossManager / BossHealth / BossController
- AudioManager
- LevelEndManager / VictoryManager

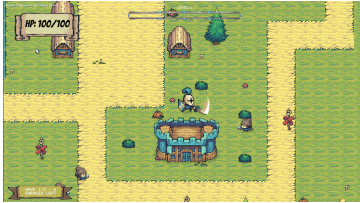
Gameplay, Features und Mechaniken

- Nahkampf, Bewegung und Sammeln bilden die Core-Loop des Spiels.
- Gold wird direkt in XP und Level-Fortschritt umgewandelt.
- Bei jedem Level-Up wählt der Spieler eines von drei Upgrades.
- Upgrades verbessern Schaden, Leben, Tempo, Defense oder Regeneration.



Level 1 und Level 2 im Vergleich

Level 1



Grüne Karte, offener Einstieg, frühe Wellen und ein lesbarer Start in die Core-Loop.

Level 2



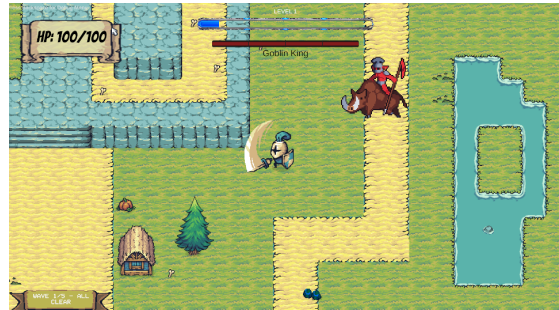
Anderes Farbklima, deutlich bedrohlichere Stimmung und Eskalation Richtung zweitem Bosskampf.

- Der Unterschied zwischen beiden Levels ist absichtlich visuell klar: Level 1 wirkt klassisch-abenteuerlich, Level 2 harscher und boss-orientierter.
- So wird Fortschritt nicht nur über Zahlen, sondern auch über Umgebung, Ton und Kampfintensität vermittelt.

- Gegner erscheinen in klar aufgebauten Wellen mit Countdown.
- Unterschiedliche Gegnertypen sorgen für Abwechslung im Kampf.
- Erst wenn alle Gegner besiegt sind, startet die nächste Welle.
- Sichtbare Standardgegner sind Pawn und Lancer.

Boss-System und Bosskämpfe

- Bosskämpfe werden mit Intro, Kamera-Fokus und Boss-Musik inszeniert.
- Beide Bosse haben mehrere Kampfphasen und eigene Angriffe.
- Goblin King setzt auf Nahkampf, Projektil und Charge.
- Vampire Bat ist der stärkere Endboss mit aggressiverem Move-Set.



- UI zeigt HP, XP, Wave-Status und Bossleiste klar und direkt.
- Musik und Soundeffekte unterstützen Level-Up, Bosskampf und Victory.
- Nach Level 1 folgt der Übergang in Level 2.
- Der Victory-Screen bildet den klaren Abschluss des Spiels.



Level-Fortschritt über mehrere Szenen

- `PlayerSessionData` speichert Level, XP, HP, Angriff, Speed, Defense und Regeneration zwischen Szenen.
- Dadurch bleibt die Progression von Level 1 nach Level 2 erhalten.
- Das unterstützt die Spielidee, dass Level-Ups und Upgrades spurbare Auswirkungen auf den späteren Bosskampf haben.
- Technisch ist das eine einfache und robuste Losung ohne externes Save-System.

Aufgetretene Probleme und Lösungen

1. Bosskampf-Inszenierung statt abruptem Spawn

Problem: Ein normaler Spawn wirkte technisch und unspektakular.

Lösung: BossManager kombiniert Kamera-Fokus, Overlay-Blinken, Spawn-SFX, Spawn-Delay und Boss-Musik zu einer kurzen Intro-Sequenz.

2. Fortschritt über zwei Level hinweg erhalten

Problem: Werte wie Level, XP und Upgrades durften beim Szenenwechsel nicht verloren gehen.

Lösung: PlayerSessionData speichert die relevanten Player-Werte vor dem Levelwechsel und stellt sie im nächsten Level wieder her.

3. Lesbarkeit im Kampf trotz vieler Systeme

Problem: Wellen, XP, Boss-HP, Audio und Upgrade-Auswahl konnten schnell unübersichtlich werden.

Lösung: Trennung in Systeme wie WaveUI, LevelUI, UpgradeSelectionManager und AudioManager; zusätzlich pausiert die Upgrade-Auswahl das Spiel.

4. Korrektes Wave-Ende

Problem: Eine Welle darf erst enden, wenn alles gespawnt und alles besiegt wurde.

Lösung: WaveManager verwaltet getrennt enemiesToSpawn und enemiesAlive und wartet auf beide Bedingungen.

Code-basierte Einordnung der Teamleistung

Person	Beitrag im Projekt
Bilgehan	Kernskripte für Spieler, Level, Wellen, Gegner, Session-Fortschritt sowie Level 1 und Level 2 als spielbare Struktur.
Fabian	Unterstützung bei Bosslogik, Upgrade-System, Level-up-Design und allgemeiner Gameplay-Feinabstimmung.
Selim Berk Tan	UI-Finishing, Victory- und Endscreen, Soundeffekte und Abschlussfluss des Spiels.

- Dungeon Surge besitzt eine klare Gameplay-Schleife aus Kampf, Ressourcen, XP, Upgrades und Boss-Fortschritt.
- Das Projekt kombiniert spielerische Lesbarkeit mit modularen Unity-Systemen für Gegner, Bosse, UI und Audio.
- Besonders stark ist die Verbindung aus Level-Progression, Upgrade-System und Boss-Inszenierung über zwei Level hinweg.

Fragen?